Healing Pipeline — Premature-Exit Audit

Scope: every place from `ExecutionEngine` → `Validation` where healing can terminate, short-circuit, or refuse a locator swap (`return` / `continue` / `break` / `report_only` / `skip` / `healable=false`).

Method: static read of the actual code paths. No code was changed.

Legend for “Should it be a hard stop or an advisor?”
- **HARD STOP** = correct to terminate; locator healing genuinely cannot help.
- **ADVISOR** = should not terminate the pipeline; it should emit a signal/score and let later layers still try a grounded locator swap.
- **GREY** = depends on evidence quality; currently a stop, arguably an advisor.

Stage 0 — ExecutionEngine (`src/core/execution-engine.ts`)

#	Location	Exit	Evidence that triggers it	Could locator healing still succeed?	Verdict
E1	`installDependencies` throws (`no package.json` , `npm install failed x2` , `node_modules missing`)	throw → job fails	filesystem / npm state	No — nothing can run	**HARD STOP**
E2	`run` / `runAsync` timeout → `exitCode 124` , often **no results file**	killed at `HEALING_RERUN_TIMEOUT_MS` (120s)	wall-clock only	**Maybe** — a slow page can look like a timeout; the original break may still be a locator	**GREY** — currently indistinguishable from a real hang
E3	`exitCode 127` (command not found)	logged, returns empty-ish result	launcher/PATH	No	**HARD STOP** (but should be labeled `environment` , not flow into `framework` / `unknown`)
E4	No `test-results.json` produced and stdout isn't JSON	returns result with empty report	crash before reporter init (xvfb/xauth, OOM, bad flag)	**N/A for this run**, but it **must not be read as the test's verdict**	**GREY** — see Cross-cutting #1

Key risk: E2 + E4 both yield "a run with no/again-failing result" that downstream cannot distinguish from "the locator is still broken." This is the exact ambiguity that turned execution 118 into a `framework` /report-only.

Stage 1 — ArtifactCollector (`src/core/artifact-collector.ts`)

#	Location	Exit	Evidence	Could locator healing still succeed?	Verdict
A1	`collect()` throws if `test-results.json` missing (L68)	throw	file missing	No	**HARD STOP** (correct — but caller should treat as “no verdict”, not “framework”)
A2	`walkSuites` skips `passed` / `skipped` results (L96)	`continue`	result status	n/a	**CORRECT**
A3	A failure with **no parseable locator** → `failed_locator: null` (L211)	not an exit itself, but **starves** every grounded advisor	Playwright didn’t echo a locator AND no Page-Object resolution	**Yes** — App Profile / DOM could still ground a selector from the line or URL	**ADVISOR** — null locator should downgrade confidence, not silently disable healing
A4	`findActualSourceLocation` falls back to spec file when no PO frame found (L343)	wrong `file_path`	stack has no PO frame	Heal may target the wrong file → validation “locator not in file”	**GREY** — misresolution masquerades as “unhealable”
A5	Top-level load errors live in `errors[]`, **not** walked by `collect()` (L456 helper exists but `collect` ignores it)	0 artifacts collected	spec throws at import / global-setup fails	No (test never ran)	**HARD STOP**, but currently **silent** — surfaces as “0 failures”

Key risk: A3 is the quiet one — a null `failed_locator` doesn’t stop the pipeline, it defangs it: rule/pattern/validation/DOM all have nothing to anchor

on, so the only survivor is the AI (or nothing), and the diagnosis slides toward `unknown` → report_only.

Stage 2 — FailureAnalyzer + FailureClassifier (`failure-analyzer.ts`, `failure-classifier.ts`)

#	Location	Exit / verdict	Evidence	Could locator healing still succeed?	Verdict
C1	`detectFailureType` → `unknown` (analyzer L130)	not an exit; sets category `unknown`	error text matched none of locator/timeout/assert/nav patterns	**Often yes** — "target closed" after a real locator wait still has `waiting for locator(...)` earlier in the joined message	**GREY** — the regex is first-match-wins on a joined multi-error string
C2	`looksLikeFrameworkFailure` → `category=framework`, `confidence 0.8` (classifier L230)	report_only downstream	`target closed/crashed`, `protocol error`, `browserType.launch`, `executable doesn't exist`	**Sometimes** — a crash can happen after a locator timeout; the framework signal then masks a real locator break	**GREY → ADVISOR** — framework should be a signal, not a terminal category, **unless** it's the ONLY evidence
C3	`looksLikeApiFailure` / `looksLikeEnvironmentFailure` → `api` / `environment` (L233-239)	report_only	request/status/env-var regex	Rarely	**GREY** — same masking risk as C2
C4	`healableByLocatorSwap = category==='locator' && !!locator` (L286)	gates all swap healing	category + locator presence	**This is THE gate** — any mis-categorization here kills healing	**ADVISOR territory** — see Cross-cutting #2
C5	`refineDiagnosisWithEvidence`: `!`	evidence ignored if `loc-`	EvidenceCollector produced no locatorState	**Yes** — exactly execution-118's case: `locat-`	**GREY** — no evidence ⇒ classifier should defer,

#	Location	Exit / verdict	Evidence	Could locator healing still succeed?	Verdict
	`ls.exists` only fires when `ls.source !== 'unknown'` (L372)	`ator-State=null`	(no DOM/live probe)	`orState: null` ⇒ evidence stage is a no-op ⇒ parser verdict stands	not let a weak parser verdict become final
C6	`strategyFor-Category` : everything except locator/timing → `report_only` (L327)	report_only	category	n/a (consequence of C1-C4)	**Consequence**

Key risk (matches your hypothesis, but inverts the blame): the evidence refinement (C5) is **disabled whenever `locatorState` is null** — and `locatorState` is null precisely when there was no DOM snapshot / live probe (framework crash, fast failure). So a weak parser verdict (`unknown` → `framework`) is never corrected by evidence. The shape isn't corrupted; the evidence stage is skipped, leaving the parser's first guess as the final word.

Stage 3 — Strategy Router (`healing-strategy-router.ts`)

#	Location	Exit	Evidence	Could locator healing still succeed?	Verdict
R1	Guard 1: `confidence < 0.5 && category! =='locator'` → report_only (L78)	report_only	low confidence	**Yes** — low confidence is a reason to gather more, not to refuse	**ADVISOR**
R2	`case 'locator'` but `! healableBy-LocatorSwap || !locator` → report_only (L103)	report_only	no concrete locator	Maybe (App Profile could ground from URL)	**GREY**
R3	`assertion` → report_only (L132)	report_only	element found, value mismatch	No (real product defect)	**HARD STOP** ✓
R4	`navigation` → report_only (L137)	report_only	net::ERR, navigation	No	**HARD STOP** ✓
R5	`api` → report_only (L142)	report_only	request/status	No	**HARD STOP** ✓
R6	`environment` → report_only (L147)	report_only	env/cred/permission	No	**HARD STOP** ✓
R7	`framework` → report_only (L152)	report_only	C2 signal	**Sometimes** (C2 masking)	**GREY → ADVISOR**
R8	`unknown` /default → report_only (L157)	report_only	nothing matched	**Often** — “unclassified” ≠ “unhealable”	**ADVISOR** — biggest false-negative source

Key risk: R7 and R8 are terminal today. Together they convert every ambiguous or crash-tinged failure into report_only **before** the App Profile / Repo Intelligence advisors ever get a chance to ground a selector. This is the architectural seam you’re pointing at.

Stage 4 — Worker routing (`src/api/server.ts`, `createHealingWorker`)

#	Location	Exit	Evidence	Could locator healing still succeed?	Verdict
W1	`isCancelled()` → break (L801, 1382, 1487)	break	user cancel	n/a	**HARD STOP** ✓
W2	`jobBudgetExhausted()` / `testBudgetExhausted()` → break (L805, 1386, 1487)	break	wall-clock	**Yes** — ran out of time, not options	**HARD STOP** (acceptable) but should be reported as `timed_out`, not `not_healed`
W3	`failureType==='navigation'` → skip (L1096)	skip+restore	analyzer type	No	**HARD STOP** ✓
W4	`failureType==='assertion'||'timeout'` → wait-inject or skip (L1102)	skip (locator loop never entered)	analyzer type	**timeout: maybe** — generic timeout can hide a locator wait	**GREY** for `timeout`
W5	`strategyPlan && !shouldAttemptLocatorHealing` → skip (L1186)	skip+restore	router verdict (R1-R8)	inherits all router GREYs	**GREY** — this is where R7/R8 become a real skip
W6	Deterministic `no_profile` → trail.skip, fall through (L1355)	**not terminal** (falls to intelligent pipeline)	no App Profile	Yes (intelligent pipeline still runs)	**CORRECT** ✓
W7	`ranked.candidates.length`	break	no advisor produced a syntactically	No (truly nothing to try)	**HARD STOP** (but A3 star-

#	Location	Exit	Evidence	Could locator healing still succeed?	Verdict
	`===0` → break (L1461)		valid candidate		vation can cause this)
W8	`acceptCandidate` pre-check reject → continue (L1541)	continue	static acceptance	tries next candidate	**CORRECT** ✓ (per-candidate, not pipeline)
W9	`validationLayer.validate` not approved → continue (L1556)	continue	static validation	tries next candidate	**CORRECT** ✓
W10	Confirmation rerun crashed before tests → revert+continue (L1817)	revert	no results file, exit≠0	**Yes** — environment crash reverted a possibly-good fix	**GREY** — environment failure should not count as candidate failure
W11	Same locator still failing → revert+continue (L1930)	revert	rerun still red on same locator	tries next candidate	**CORRECT** ✓
W12	`!locatorFixed` after retries → break (L1980)	break	exhausted candidates	No	**HARD STOP** ✓

Cross-cutting findings (the architectural ones)

CC-1 — “No verdict” is silently treated as “the test’s verdict”

E2/E4/A1/W10 all produce absence of a passing result. The pipeline collapses two very different facts into one:

- “the locator is still broken” (a real healing signal), and
- “the run never produced a trustworthy result” (an environment fact).
A framework crash / timeout / missing-report should yield **`inconclusive`**, not `framework` / `not_healed`. Today there is no `inconclusive` state.

CC-2 — `healableByLocatorSwap` is a single chokepoint, set by the weakest evidence

Every grounded advisor (Repo Intelligence, App Profile, DOM Memory) is gated behind one boolean computed in the classifier from regex + (optional) evidence. When evidence is absent (C5) the regex wins. In an advisor model this boolean should be **one advisor’s opinion**, not the gate for all of them.

CC-3 — Terminal categories run BEFORE grounded advisors

`framework` (R7) and `unknown` (R8) terminate at the router — upstream of the App Profile / Repo Intelligence resolution. So the very intelligence that could disprove the “framework/unknown” guess never runs. Order is inverted relative to the product promise (“Repo Intelligence first”).

CC-4 — First-match-wins on a JOINED multi-error string

`error_message` is `errors[].message.join('\n\n')`. `detectFailureType` and the `looksLike*` helpers scan the whole blob; whichever pattern hits first wins. A genuine `waiting for locator('#x')` followed by a teardown `Target closed` can be classified `framework` because the framework regex matched somewhere in the blob.

Recommended exit taxonomy (no code yet — for discussion)

Exit class	Examples	Desired behavior
HARD STOP	assertion, navigation, api, environment, cancel, no-package.json	terminate, report honestly (these are correct today)
INCONCLUSIVE (new)	timeout w/ no results, framework crash, missing report, rerun crashed-before-tests	do NOT label as test verdict; retry once clean OR mark inconclusive; never count as “locator unhealable”
ADVISOR (downgrade from stop)	`unknown`, `framework`-as-sole-signal, low-confidence, null locator	emit signal + score, STILL run Repo Intelligence + App Profile grounding before giving up
PER-CANDIDATE (already correct)	acceptance/validation reject, same-locator-still-failing	continue to next candidate

---

Bottom line

- The **hard stops for assertion/navigation/api/environment are correct.**
- The **dangerous exits are the "ambiguous → terminal" ones**: `framework` (R7), `unknown` (R8), low-confidence (R1), and every "no trustworthy result" path (E2/E4/A1/W10), plus the **null-locator starvation** (A3) and the **evidence-skipped-when-locatorState-null** behavior (C5).
- All of these terminate **before** Repo Intelligence / App Profile grounding — which is exactly the inversion an advisor model would fix: let each advisor vote, treat "no result" as inconclusive (not failure), and only declare "unhealable" after the grounded advisors have actually run.